

CSC2106: IoT Protocols and Networks

Design Review Report

Abstract

This project presents the design and implementation of a distributed multiplayer button-press game where multiple players compete to press a button first and the fastest player wins. The core challenge addressed is ensuring fair winner determination under real-world wireless network conditions, where network congestion and varying transmission delays mean that the order in which the server receives signals does not necessarily reflect the true order in which buttons were pressed. To solve this, each device independently measures elapsed time from a synchronized GO signal to the button press, allowing the server to compare local reaction times rather than relying on packet arrival order. Multi-hop communication via ESP-NOW and a simplified AODV-like routing is supported, enabling participation across extended ranges. Preliminary evaluation indicates end-to-end latency under 409 ms in direct-link scenarios, with relay hops introducing an additional 5–15 ms per hop. Fairness was preserved because the winner was decided using local reaction time even under relay-based communication, and invalid or replayed packets were rejected during testing. Testing was conducted at a small scale with up to 5 devices; scalability beyond 20 nodes is constrained by ESP-NOW's encrypted peer limit. These findings suggest the timestamp-based fairness mechanism is effective under controlled conditions, though further testing under high-congestion and large-scale deployments remains future work.

Section I: Introduction, Problem Statement and Objectives

Problem Statement: How can a distributed multi-hop wireless IoT system provide fair and low-latency first-responder determination, while maintaining basic security and reliable operation under controlled multi-hop network conditions?

In competitive quiz-based applications, fairness in determining the first responder is critical. Traditional wireless input systems often rely on server packet arrival time to decide the winner. However, in real-time IoT environments, packet arrival order does not necessarily reflect the true physical order of button presses due to network latency, jitter, congestion, and routing dynamics.

The originally identified problem was the absence of a reliable, low-latency, and user-friendly wireless input system that can support real-time interaction while accurately determining the first responder in classroom and lab-style wireless environments. Existing systems often suffer from communication delays, inconsistent response detection, limited security provisions, and poor scalability, reducing fairness in competitive environments.

In wireless communication systems using BLE or Wi-Fi, differences in Media Access Control (MAC) layer mechanisms introduce timing inconsistencies. BLE relies on frequency hopping and connection intervals, while Wi-Fi uses CSMA/CA with exponential backoff under congestion. As a result, packet transmission delay varies depending on channel conditions, interference, and device proximity. This creates a fundamental issue: the participant whose packet arrives first at the server may not necessarily be the participant who pressed first.

In the current system design, the problem is extended to a distributed multi-hop wireless network using ESP-NOW and a simplified AODV-like (Ad-hoc On-Demand Distance Vector) routing. Nodes that are out of direct communication range must relay packets through intermediate nodes, introducing hop-dependent latency variation. This means fairness cannot depend on physical proximity to the server or routing topology.

The core identified challenge is therefore reframed as a distributed systems problem: how to ensure fair real-time arbitration of button press events in a multi-hop, congestion-prone wireless IoT network where packet arrival time is unreliable and routing paths are dynamic.

Furthermore, security considerations must be addressed. Unauthorized packet injection, replay attacks, or spoofed button presses could compromise fairness. However, stronger security mechanisms may increase packet size and processing delay, creating a security-latency trade-off. To manage this trade-off, the prototype uses lightweight security controls such as ESP-NOW AES-128 unicast encryption, shared-secret authentication, and replay suppression. These mechanisms are intended to reduce basic spoofing and replay attacks with low overhead, but they are evaluated only in controlled tests and are not claimed to provide comprehensive protection against advanced attackers.

The primary objective of this project is to design, develop, and evaluate a distributed multi-hop wireless button system capable of millisecond-accurate and fair winner determination under dynamic network conditions.

To achieve this, the project sets the following objectives:

1. Design a Fair Timestamp-Based Mechanism

Each button node will locally measure the elapsed time between receiving a synchronized GO signal and the physical button press. The central server will determine the winner based on measured reaction time rather than packet arrival time, eliminating bias introduced by transmission delay.

Unlike the original proposal which focused on cross-protocol BLE and Wi-Fi arbitration, the updated system adopts a unified multi-hop wireless approach that removes dependence on fixed infrastructure and allows fairness to be evaluated in a distributed topology.

2. Support Multi-Hop Communication

The system will support communication between nodes that are not in direct transmission range of the central server. This ensures scalability and flexibility in dynamic environments where device placement varies.

3. Develop an Efficient Communication Framework

A lightweight communication structure will be designed to ensure low transmission overhead while supporting fairness logic, event identification, and network coordination.

4. Ensure Congestion-Resilient Fairness

The system will be evaluated under conditions of simultaneous button presses and increased network traffic to verify that routing variability and congestion do not affect winner determination.

The project aims to demonstrate that fairness can be evaluated across the tested relay and simultaneous-press scenarios within the prototype, though full validation across all topologies is subject to the constraints of a 5-device testbed.

5. Implement Lightweight Security Controls

The system will incorporate device authentication and replay protection mechanisms to prevent unauthorized participation and ensure the integrity of button press events, while maintaining minimal impact on latency.

6. Evaluate System Performance and Scalability

The project will evaluate system responsiveness, fairness accuracy, and reliability under varying network conditions within the scope of a 10-device prototype to validate that human reaction time remains the determining factor in competition outcomes.

The completed system will demonstrate that fair real-time event arbitration in wireless IoT networks can be achieved without relying on packet arrival order or global clock synchronization. By shifting decision logic from server-receive timing to locally measured reaction intervals, the design decouples fairness from transport-layer variability.

Additionally, by integrating multi-hop wireless routing within a real-time competitive application, the project aims to show how dynamic network design can support fair, latency-resilient event arbitration, extending the system beyond a basic quiz buzzer into a small-scale distributed IoT prototype.

Section II: Literature Review

1. What is the current state of knowledge or technology in this field?

To ensure fairness in a competitive quiz system, the connection between wireless buttons must prioritize ultra low latency and synchronization. The field currently relies on the ESP32 ecosystem, a 32-bit microcontroller standard for IoT due to its integrated Wi-Fi and Bluetooth stacks[1]. While Wi-Fi remains the baseline for high throughput local infrastructure and Bluetooth (BLE 5.2) serves as the standard for Personal Area Networks (PAN), both protocols exhibit a "latency gap".[1] Current research identifies that traditional handshakes and infrastructure dependencies in these protocols often hinder the real time, deterministic response timing required for synchronized button triggering events [1].

2. What are the most significant research findings or technological developments related to this topic?

The most significant development is the ESP-NOW protocol, which enables "connectionless" communication. Unlike standard WiFi, it bypasses time-consuming handshakes, allowing peer-to-peer data transmission in a significantly shorter window. Research shows it supports secure, 250-byte encrypted payloads[1], making it a breakthrough for event driven tasks. Additionally, the shift toward integrated hardware like the M5Stack/M5Stick has streamlined "Edge" development, allowing developers to couple low latency protocols directly with built-in user interfaces for more accurate event ordering and first responder arbitration[1].

3. What methodologies have been commonly used in previous studies, and how effective were they?

Previous studies have utilized hardware-assisted profiling to measure wireless latency with microsecond precision[2]. A common methodology involves toggling General Purpose Input/Output (GPIO) pins at critical stages of the software stack, such as packet generation on the application core and reception on the network core, monitored via a digital oscilloscope[2]. This approach is highly effective for characterizing protocol performance, such as identifying that disabling Cyclic Redundancy Checks (CRC) and using manual transmission modes can reduce device-to-device latency to approximately 500 μ s [2]. These empirical methods allow researchers to isolate delays caused by inter-process communication versus actual radio transmission[2]. This study adopts the hardware-assisted profiling principles established by Krull et al. [2], specifically utilizing the M5Stick's internal high-resolution timers to replicate the precision of GPIO-based event logging without requiring external oscilloscopes in a mobile environment.

4. What are the limitations or shortcomings of existing solutions or approaches?

Existing solutions often rely on MCU channel-based architectures, which are structurally limited by I/O resources and instruction cycles, leading to poor scalability for large groups [3]. Furthermore, many systems utilize fixed Resistor-Capacitor (RC) timing, which suffers from temperature drift, component tolerance errors, and a lack of configurability for different question difficulties [3]. Traditional manual registration also creates bottlenecks, with low throughput (15–30s per person) and high entry error rates (3–5%). These hardware constraints and fixed-timing limitations compromise fairness and prevent the adoption of flexible, team-based instructional designs [3].

5. How do the findings from previous research relate to the objectives of this study?

Based on the comparative study by Eridani, ESP-NOW demonstrates better latency and range characteristics for low payload event triggering compared to the overhead heavy handshakes of Wi-Fi and BLE [1]. The achievement of sub-millisecond, deterministic synchronization through hardware level profiling confirms that a "Go packet" methodology, where buttons calculate response duration locally, is superior to server-side arbitration [2]. This approach ensures fairness by decoupling user response time from network transmission latency, allowing for accurate first responder detection even across varying distances in a Mesh topology. Furthermore, moving from fixed RC-timing circuits to a software configurable, MCU-based platform like the M5Stick aligns with findings that digital logic architectures improve scalability [3]. Together, these studies provide the empirical foundation for building a high-precision quiz system using edge-based timestamping.

Section III: Design

1. Overall Architecture of the System

a) System Components and Devices

The system uses a distributed wireless IoT architecture built with ESP32-based M5Stack/M5Stick devices. It consists of three components: a central server node, player nodes, and optional relay nodes.

1. **Central Server Nodes:** Coordinates the game by broadcasting a GO signal and collecting reaction-time packets. It determines the winner by comparing recorded reaction times.
2. **Player Nodes (M5Stick device):** Each node has a physical button, a local timer, LED display. Upon receiving the GO signal, the node timer starts. When the button is pressed, reaction time is recorded and transmitted to the server.
3. **Relay Nodes:** Extend network coverage by forwarding packets from nodes outside the server's direct range, enabling multi-hop communication and flexible deployment.

b) Network Topology and Connectivity

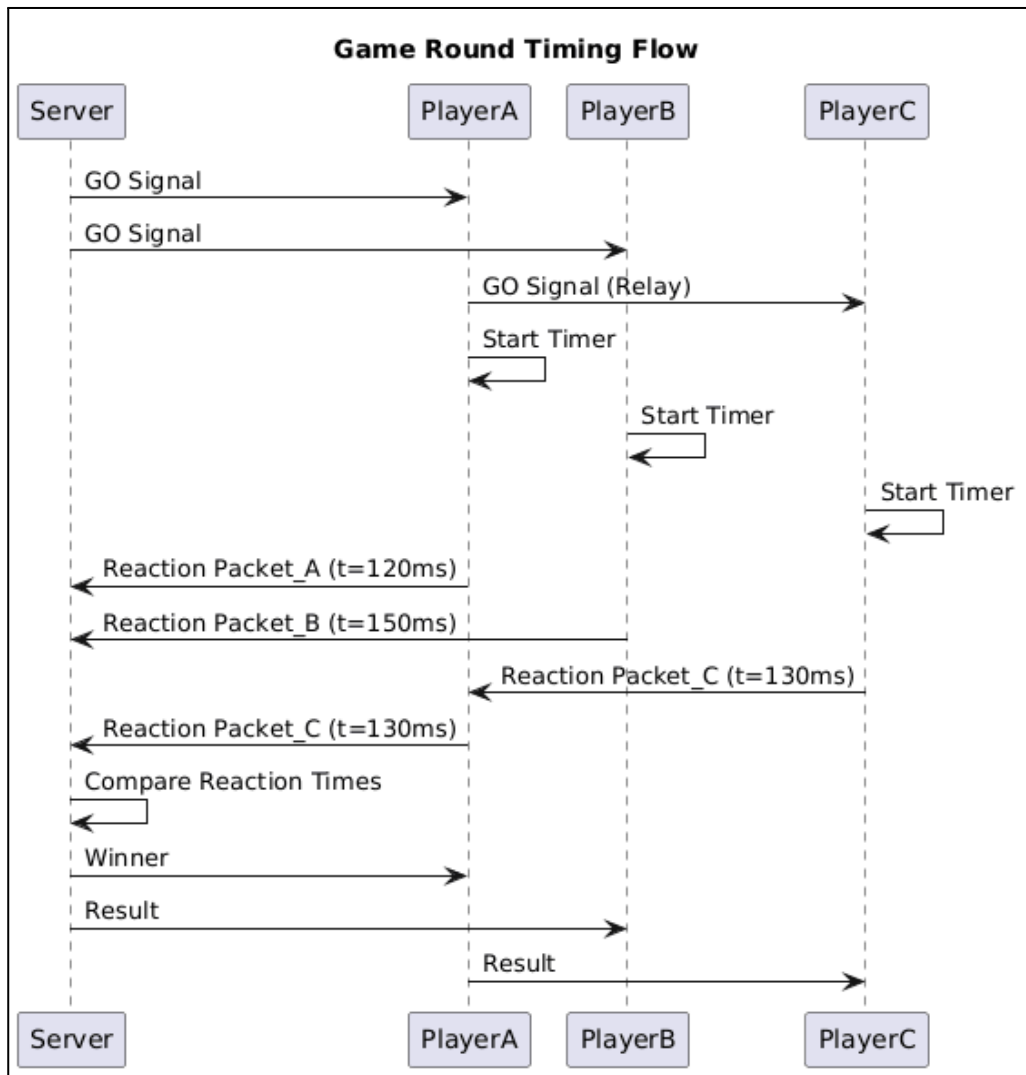
The system uses a multi-hop wireless topology based on ESP-NOW, enabling connectionless communication between ESP32 devices with minimal latency.

The server broadcast a GO signal to all nodes. Player nodes respond with packets containing:

- Device ID
- Reaction time
- Game round identifier
- Shared-secret authentication field

If the node cannot reach the server directly, packets are forwarded through relay nodes using a simplified AODV-like routing, which dynamically discovers routes and adapts to topology changes.

The following is a sequence diagram:



c) Data Collection and Communication Protocol

Data Collection is event-driven, triggered by button presses.

1. Game Start:

The server sends a GO broadcast packet to all nodes containing:

- Round ID
- Timestamp reference
- security token

Nodes begin timing upon receipt.

2. Reaction Measurement

When a button is pressed, the node calculates the elapsed reaction time and generates a response packet.

3. Transmission

Packets are sent directly or forwarded via relay nodes using a simplified AODV-like algorithm.

4. Winner Determination

The server compares all received reaction times and selects the lowest value.

This ensures fairness by relying on local timing rather than packet arrival order, which may be affected by network delays.

2. Data Storage and Processing

Data is processed locally at the server node, as the system focuses on real-time decision-making rather than long-term storage.

During each round, the server temporarily stores:

- Device ID
- Reaction time
- Round ID
- Packet timestamp
- Optional routing information

After a predefined response window (15 seconds) or when the server received all PRESS packets from players (whichever is shorter), the server:

1. Validate packet authenticity
2. Remove duplicate or replayed packets
3. Compare reaction times
4. Determines the winner
5. Broadcast the results

Clock Synchronisation and Timing Accuracy:

The system does not rely on global clock synchronisation. Instead, each node starts its timer upon receiving the GO signal.

- Propagation Delay:

Direct communication delay is typically ~5ms, while multihop routing introduces an additional 5-15ms per hop. This does not affect fairness since timing is measured locally.

- Clock Drift:

ESP32 timers may drift by $\pm 10\text{--}50\text{ppm}$, resulting in negligible error over short durations. Given that human reaction times are typically 200–300ms, this variation is insignificant.

3. User Interaction

a) Player Interaction

Players interact via the M5Stack device.

1. Waits for the GO signal
2. Presses the button as quickly as possible
3. Receives results feedback via the LED display

b) Server Interaction

The game host uses the server node to:

- Start or reset the game round
- Monitor responses
- Display the winner

No external applications are required, making the system suitable for classrooms and interactive events.

4. Security Protocols

A lightweight security model is implemented to protect fairness and system integrity.

Device Authentication

Each node is authenticated by the shared secret key, then registered by the node MAC address. The server then assigns the node a unique device ID. Subsequently, the server only accepts packets from authorized devices.

Protocol Validation

Incoming packets are checked for expected size, protocol version, and authentication validity before being processed. Invalid or incompatible packets are discarded early to reduce malformed traffic and unintended behavior.

Encryption

ESP-NOW uses AES-128 encryption for unicast to secure communication and prevent packet interception or manipulation. Furthermore, SHA256 is used to hash the packet message, providing basic data integrity.

These mechanisms ensure security while maintaining low communication latency.

Section IV: Implementation (Prototype and Testing)

1. How will the system be prototyped, and what tools or platforms will be used for development?

The prototype uses 5 M5Stack Plus devices based on the ESP32 microcontroller. The ESP32 was chosen because it provides built-in Wi-Fi, sufficient processing power, and buttons for a wireless reaction-time game.

One device acts as the server, which starts each round and collects player responses. The remaining devices act as player or relay nodes when direct communication to the server is unavailable. This allows both direct-link and multi-hop testing.

Development is carried out in Arduino IDE using the ESP32 framework. The code is organized into modules for game control, communication, timing, routing, and packet verification. ESP-NOW is used because it supports low-latency device-to-device messaging without external infrastructure.

Button input is debounced in software using a 50 ms delay. Reaction time is measured using the ESP32 `millis()` function, which provides 1 ms resolution. This is adequate for comparing human reaction times, although sub-millisecond differences cannot be resolved. Accuracy was checked through repeated rounds and by introducing intentional relay delay to a faulty node. Even when delivery delay increased, the recorded `reaction_ms` value still represented the local interval from GO reception to button press, confirming that winner selection depended on local timing rather than transmission delay. Because all nodes used the same ESP32 platform, identical timing logic, and a fixed ESP-NOW channel, timing behaviour was consistent across devices.

2. How will the IoT system be deployed and what challenges might arise during deployment, and how will they be mitigated?

The system is intended for controlled indoor environments. The server is placed centrally and player nodes are distributed around the room. Some nodes may be placed outside direct range so that relay behaviour can be tested.

Before each round, nodes register with the server. When the server sends the GO signal, each player presses the button as quickly as possible. The node records the reaction time locally and sends it back either directly or through relay nodes.

A challenge is packet collision when several devices transmit at nearly the same time. This is mitigated by keeping packets compact, adding small random delays before broadcast routing messages, and spacing server transmissions. All devices are locked to the same ESP-NOW channel to maintain communication. The prototype was tested indoors. In noisier RF conditions, packet loss may increase, so dynamic channel selection is left as future work. The design also has no server failover mechanism; if the server fails mid-round, players eventually time out. Device failure and power instability are mitigated through pre-deployment checks, spare units, and deep sleep on player nodes.

3. What will be the process for testing the IoT devices and the overall system to ensure reliability and functionality?

Testing is performed in two stages: unit testing and integration testing. Unit testing checks each device individually to confirm correct button detection, time recording, and packet transmission and reception. Integration testing connects all devices into a working network and runs repeated game rounds.

To evaluate relay behaviour, a faulty node was configured so that it could not communicate with the server directly and instead used a single relay node (1-hop path). This provided a controlled way to study routing, forwarding, and fairness under relay-based communication.

Fairness was evaluated by comparing reaction_ms values logged by the server across repeated rounds. Each player measures response time locally from GO reception to button press, so the recorded value reflects local timing rather than packet arrival order. Repeated near-simultaneous press tests were also conducted to observe behaviour under contention. Fairness across hop counts was examined by comparing direct-link players with the relay-dependent faulty node; although relay routing increased delivery delay, winner determination still depended on local reaction time. Security testing was performed by sending invalidly authenticated packets and replayed packet identifiers. Serial logs showed that such packets were rejected as authentication failures or duplicates.

4. How will success be measured or evaluated? What benchmarks or metrics will be used for evaluating the system's performance?

The system is evaluated using four criteria: fairness, latency, reliability, and scalability. Fairness is the primary metric because the system must identify the fastest responder using local reaction time rather than packet arrival order. Latency is measured from the GO signal to the result. Reliability is assessed through packet delivery behaviour across repeated rounds, while scalability is judged by observing how performance changes as more nodes and relay paths are introduced. If these criteria remain acceptable within the prototype scope, the system meets its design goals.

Section V: Results and Analysis

1. How will scalability and network performance be managed? Specifically, can it handle additional devices, users, or data without degradation in performance?

Scalability is managed by the use of a simplified AODV-like routing, as it only discovers routes when needed, saving bandwidth compared to proactive protocols that constantly map the network. This allows the addition of more player nodes without clogging the airwaves with background data. However, performance will degrade with more users, but it can be mitigated. As the number of players increases, radio

frequency (RF) collisions increase when many players click at once. However, using a randomized “back-off” time for retransmissions in the simplified AODV logic will resolve this issue.

The relay mechanism, where a player node can use other player nodes to relay messages to the central hub, also ensures scalability. But it brings up a problem where every time an M5 stick acts as a relay, it consumes a small amount of processing power and adds a typical 5 to 15 milliseconds of latency per hop. This greatly affects the network performance, as adding relays will make the time of arrival at the central hub no longer a fair metric because players that are more hops away will always appear slower than someone with fewer hops away. However, this will be mitigated by using the time taken to react to the message when the game starts instead of using the timestamp.

There is also a limit of devices when we use the ESP-NOW protocol for communication, which can theoretically handle only 20 paired peers in encrypted mode.

2. What were the key performance metrics (e.g., response time, data throughput, latency, reliability) and how did the system perform against them?

Packet Success Rate: The percentage of the number of successful packets sent to the total number of packets sent

Data throughput: Measure how well the system can handle multiple data streams at once.

Round trip latency is determined by GO travel time (server → player) + PRESS travel time (player → server).

Metric	Performance (No relay)	Performance (Relay, 1-hop)	Performance vs Human Reaction
Packet Success Rate	100%	100%	Good (Rarely miss)
Data throughput	Low (~5.7 kbps, 22	Low (~1.2 kbps, 3.8	Sufficient (game

	pkt/s avg)	pkt/s)	only needs small bursts)
Round Trip Latency	409.3 ms avg (min 154ms)	3844ms avg (min 33 ms)	Acceptable (winner determined by local timestamp, not arrival time)

3. How does the system's performance compare with similar IoT solutions?

Comparison of ESP-NOW to other protocols:

Protocol	Latency (ms)	Range
ESP-NOW	<5ms	~100m [1]
Bluetooth (BLE)	~20-50ms	~30m [1]
Zigbee (Mesh)	~30-100ms	~100m+ [4]
LoRaWAN	>1 second	10km+ [5], [6]

4. How closely did the results align with the initial objectives and goals set at the start of the project?

Initial objectives:

1. Reliable Communication

The results aligned well with this objective because the system achieved a packet success rate of 100% in both relay and non-relay conditions. This shows that the communication between devices was highly reliable during testing.

2. Range Extension Through Relay

This objective was achieved because the relay setup allowed communication even when a direct connection to the central hub was not available. The successful transmission of packets in relay mode shows that the system was able to extend its effective communication range.

3. Sufficient Data Throughput for Gameplay

The results showed that data throughput was relatively low, especially when using a relay, but it was still sufficient for the game since the system only needed to send small

bursts of data. This means the objective was met in the context of the project's actual gameplay requirements.

4. Acceptable Latency for Game Response

This objective was only partially achieved. The measured latency increased significantly when a relay was used, which means the system was not ideal for fast real-time communication. However, the game could still function because it did not depend entirely on packet arrival speed for determining the result

5. Fair Winner Determination

This objective was achieved because the winner was determined using the player's local timestamp rather than packet arrival time. As a result, network delay did not unfairly affect the final outcome, making the game fairer for all players.

6. System Scalability

The results generally aligned with this objective as the system was designed to support multiple players, and the data handling remained manageable for the game scenario. This suggests that the system was able to perform adequately for the intended number of players.

5. Were there any bottlenecks in data transmission, processing, or system response times? If so, what caused them?

There were bottlenecks in the wireless routing and retransmission. The multi-hop simplified AODV-style route discovery (RREQ/RREP with jitter) introduced startup latency before a stable route was available. During gameplay, end-to-end PRESS confirmation depends on the ACK delivery. This is such that if an ACK was lost, the player waits for timeout and route rediscovery before retrying, which increases response time. On the server side, GO packets were sent sequentially with inter-packet delay to avoid ESP-NOW buffer overflow, which added small per player latency. Overall, latency bottlenecks were moderate and primarily topology driven.

6. Were there any errors, failures, or unexpected behaviours in the system? How significant were these errors in terms of the overall system performance?

There is an unexpected behaviour in the system such that players are able to re-send duplicate PRESS when there are no GO packets received. This does not affect the overall system performance, as there is a limited number of times a player can press the button physically, and each packet is only 32 bits, minimally affecting the system. The M5StickC ran hot during prolonged use due to continuous radio activity. To mitigate this, a 30-second inactivity sleep mode was added for player nodes, reducing power consumption between rounds with no impact on gameplay. Players wake the device via Button B.

7. Were the security protocols effective in protecting the system?

The implemented security controls were effective against basic threats and accidental packet tampering. The system used ESP-NOW PMK/LMK for encrypted unicast links, per-packet authentication (HMAC-based hash), and protocol version checks, so malformed or unauthenticated packets were dropped.

However, the shared secret key used for authentication between the server and players is static and embedded in firmware. Broadcast traffic is also not protected by unicast encryption. Also, a truncated authentication tag and a lack of stronger anti-replay primitives limit protection against advanced attackers with prolonged access. However, these security concerns are not critical in a classroom and controlled environment, as the current security measures are effective.

Section VI: Conclusion

1. What were the most important findings or results from the project?

The most important finding from this project was that the distributed multi-hop wireless button game system was able to support fair and low-latency gameplay. Using ESP-NOW and AODV-style routing, the system successfully transmitted button-press data across multiple nodes and determined the winner based on true reaction time rather than packet arrival order. This shows that the project achieved its objective of building a reliable real-time IoT gaming system

2. What real-world problems does the project address or what potential does it have for future applications?

The project addresses the problem of fair real-time event arbitration in wireless networks, where packet arrival order is an unreliable indicator of true event order. This is directly applicable to competitive quiz systems, classroom response tools, and interactive event buzzers. Beyond these, the timestamp-based fairness approach could extend to any distributed IoT scenario requiring accurate first-event detection and even such as industrial trigger systems, emergency response coordination, or multiplayer gaming applications where fairness and low latency are critical.

3. How does this project contribute to the broader field of IoT?

The project demonstrates that fair real-time arbitration in a multi-hop wireless IoT network can be achieved without global clock synchronisation or reliance on packet arrival order. By combining ESP-NOW's low-latency connectionless communication with the simplified AODV dynamic routing and edge-based timestamping, the system shows how lightweight distributed design can decouple application-level fairness from transport-layer variability — a relevant contribution for resource-constrained IoT deployments.

4. How can this project be used as a foundation for more advanced or scalable IoT solutions?

The system architecture features a modular codebase that isolates game control, communication, routing, and authentication into independent elements, establishing a versatile framework suitable for a variety of real-time ESP32-based IoT use cases. By adapting the simplified AODV-like routing protocol and the ESP-NOW communication layer, this design can be expanded to support extensive mesh sensor networks. Furthermore, the integrated authentication and replay protection protocols provide a resource-efficient security foundation that can be readily applied to other event-driven IoT environments.

5. Based on your findings, what future work would you recommend?

Multiple avenues for future enhancement have been identified. To bolster reliability within noisy RF settings, the implementation of dynamic channel selection should be investigated. The system's resilience against single-point failures could be strengthened by introducing a server failover or leader election protocol. To more accurately resolve near-simultaneous button activations, timer precision could be transitioned from milliseconds to microseconds via the `esp_timer_get_time()` function. Lastly, to support larger-scale deployments beyond the 20-peer encrypted cap of ESP-NOW, further research into hybrid architectures or alternative communication protocols is recommended.

Section VII: References

When writing your report, please ensure that all references are formatted according to the IEEE citation style. Number the references in the order they appear in the text and do not use footnotes for citation

1. D. Eridani, A. F. Rochim, and F. N. Cesara, "Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols based on Range, Transmission Speed, Latency, Energy Usage and Barrier Resistance," in *2021 8th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE)*
2. N. Krull, L. Schulthess, M. Magno, L. Benini, and C. Leitner, "Enhanced ShockBurst (ESB) for Low-Latency Wireless Command Broadcasting in Multi-Node Systems," in *2025 IEEE 21st International Conference on Body Sensor Networks (BSN)*, Los Angeles, CA, USA, Nov. 2025
3. J. Li, W. Bian, Y. Diao, T. Zou, X. Yang, and B. Kang, "A QR-Enabled Multi-Participant Quiz System for Educational Settings with Configurable Timing," *Appl. Syst. Innov.*, vol. 8, no. 6, p. 158, Oct. 2025
4. Silicon Labs, "Zigbee Mesh Network Performance," [Online]. Available: <https://docs.silabs.com/zigbee/latest/zigbee-mesh-network-performance/05-summary>. [Accessed: Apr. 5, 2026].

5. LoRa Alliance, "What is LoRaWAN® Specification," [Online]. Available:
<https://lora-alliance.org/about-lorawan-old/>. [Accessed: Apr. 5, 2026].
6. LoRa Alliance Technical Marketing Workgroup, "What is LoRaWAN®: A technical overview of LoRa® and LoRaWAN®," Nov. 2015. [Online]. Available:
<https://lora-alliance.org/wp-content/uploads/2020/11/what-is-lorawan.pdf>.
[Accessed: Apr. 5, 2026].